

The Render Graph Architecture: A Deep Dive into Modern Rendering Pipelines

Section 1: An Introduction to the Render Graph Paradigm

The evolution of real-time computer graphics is characterized by a continuous pursuit of greater visual fidelity and performance. This pursuit has led to profound shifts in the design of both graphics hardware and the Application Programming Interfaces (APIs) that provide access to it. The most significant recent shift has been the transition from driver-abstracted, immediate-mode APIs to explicit, low-level APIs. This transition, while unlocking unprecedented performance potential, introduced a new class of complexity that necessitated a corresponding evolution in rendering architecture. The render graph, also known as the frame graph, emerged as the definitive architectural pattern to manage this complexity, enabling developers to harness the power of modern GPUs efficiently and maintainably. This section will explore the challenges that precipitated the rise of the render graph, define its core principles, and establish its historical context within the games and real-time graphics industry.

1.1 The Modern Rendering Challenge: A Paradigm Shift in API Design

The relationship between a rendering application and the graphics driver has fundamentally changed over the past decade. Understanding this change is critical to appreciating the necessity and elegance of the render graph architecture. The previous generation of APIs created a high-level abstraction, whereas the current generation exposes the low-level mechanics of the hardware, creating a new set of responsibilities for the application developer.

The Legacy of Immediate-Mode APIs

Legacy graphics APIs, such as Microsoft's DirectX 11 and earlier versions of OpenGL, operated predominantly in what is known as an "immediate mode".¹ In this paradigm, the application issues rendering commands procedurally, one after another, and the graphics driver is largely

responsible for managing the consequences of these commands.² When an application requested to draw an object, the driver would internally track resource dependencies, manage memory hazards, insert synchronization primitives (fences or barriers), and transition resource states to ensure that the commands executed correctly on the GPU.⁴ This model offered a significant advantage in terms of developer convenience. Programmers could focus on the logic of what to draw, largely unburdened by the intricate details of GPU pipeline synchronization.⁵ However, this convenience came at a cost. The driver's management was a "black box," relying on complex and often conservative heuristics to guess the developer's intent.⁴ This opacity could lead to unexpected performance bottlenecks, such as pipeline stalls, and limited the potential for fine-grained, application-specific optimization. The driver's need to handle every possible edge case often hampered parallelism and added significant CPU overhead, a problem that became increasingly acute as CPUs gained more cores and scenes grew in complexity.⁴

The Rise of Explicit APIs

Modern graphics APIs, including Vulkan, DirectX 12, and Metal 2, represent a fundamental philosophical shift. They are "explicit" APIs, meaning the burden of low-level GPU management has been transferred from the driver to the application.⁴ This architectural change was a direct response to the limitations of the older model, designed to eliminate driver overhead and give developers direct, predictable control over the hardware.

With these new APIs, the application is now responsible for tasks that were previously hidden:

- **Resource State Management:** Explicitly tracking and transitioning the state of resources (e.g., from a render target to a shader-readable texture).⁴
- **Synchronization:** Manually placing memory barriers to resolve data hazards, such as ensuring a write operation completes before a subsequent read operation begins (a "write-after-read" or WAR hazard).⁸
- **Memory Management:** Managing memory allocation, including the potential for aliasing physical memory to different logical resources.⁹
- **Queue Synchronization:** Explicitly managing dependencies between different hardware queues, such as the graphics, compute, and copy queues, using primitives like fences and semaphores.⁴

While this explicit control unlocks the full potential of modern GPUs, it also introduces a formidable level of complexity. A large-scale rendering engine for a AAA game might involve hundreds of distinct rendering passes, from shadow map generation and G-buffer rendering to lighting, post-processing, and UI rendering.⁸ Manually managing the web of dependencies, barriers, and state transitions for such a system is not only extraordinarily difficult but also incredibly error-prone, often leading to subtle bugs, visual artifacts, or GPU crashes that are notoriously difficult to debug.⁸ The code becomes brittle, where adding or removing a single pass can have cascading effects that require careful, manual adjustments throughout the

rendering pipeline.

The Need for a New Abstraction

The shift to explicit APIs created an architectural vacuum. The "global knowledge" of the frame's workload, which was once held implicitly within the driver, was now fragmented across the application's codebase. A new, higher-level abstraction was needed to reclaim this global context. This system would need to allow individual rendering features to be developed in a modular, self-contained way, where each module could reason locally about its specific inputs and outputs.⁸ Simultaneously, the system would need to aggregate this local information to form a complete, holistic view of the entire frame. This global knowledge is the prerequisite for performing the complex, whole-frame optimizations that modern hardware demands, such as minimizing memory usage, scheduling work for maximum parallelism, and inserting the most efficient and precise synchronization barriers.⁴ The render graph architecture emerged as the ideal solution to fill this vacuum, providing a structured, declarative framework for describing and optimizing the entire rendering process.

1.2 The Render Graph Solution: A Declarative Approach to Frame Rendering

At its core, a render graph (or frame graph, the terms are used interchangeably) is a high-level, intermediate representation of all the rendering work required to produce a single frame.⁷ This representation takes the form of a directed acyclic graph (DAG), a mathematical structure well-suited for modeling workflows with dependencies.⁷ In this graph, nodes represent units of work (known as "passes"), and the directed edges connecting them represent the resources (such as textures or buffers) that are produced by one pass and consumed by another.⁶

The fundamental principle of the render graph is a shift from a procedural, immediate-mode style of programming to a declarative one.¹⁴ Instead of issuing GPU commands as they are encountered in the code (the procedural "how"), the application first builds a data structure—the graph—that describes the entire frame's intended workload (the declarative "what").¹ The rendering logic is deferred. Only after the entire graph is defined does the system compile it. During this compilation step, the system analyzes the complete set of passes and their inter-dependencies to make intelligent, global optimizations. Finally, it executes the optimized graph by translating it into a sequence of low-level commands for the GPU.⁶ This "record, compile, execute" lifecycle provides the system with the holistic, frame-wide context that is necessary for efficient rendering on modern, explicit APIs.

1.3 Historical Context: The Influence of Frostbite's "FrameGraph"

While the conceptual underpinnings of graph-based task scheduling are not new, the popularization and widespread adoption of the render graph architecture in the real-time graphics industry can be traced to a specific, seminal event: the "FrameGraph: Extensible Rendering Architecture in Frostbite" presentation by Yuriy O'Donnell of Electronic Arts at the Game Developers Conference (GDC) in 2017.⁸

This presentation detailed how the Frostbite engine, used across a wide variety of AAA games, adopted a graph-based architecture to solve the challenges of building a modular, extensible, and efficient renderer.¹⁶ The talk articulated the core benefits of automated resource management, simplified dependency tracking, and the ability to implement rendering features in a decoupled manner.¹⁶

The impact of this presentation was immediate and profound. It provided a clear, production-proven blueprint for taming the complexity of DirectX 12 and other explicit APIs. In the years following, the adoption of render graph architectures became a de facto industry standard for high-end game engines.⁹ Prominent examples include Unreal Engine's "Rendering Dependency Graph" (RDG), Ubisoft's Anvil engine, and Unity's integration of a render graph system into its Scriptable Render Pipelines.⁴ This rapid, industry-wide convergence demonstrates the architecture's effectiveness in addressing the fundamental challenges of modern graphics programming.

Section 2: Deconstructing the Render Graph: Core Components and Principles

To understand how a render graph achieves its goals of optimization and maintainability, it is essential to first deconstruct its fundamental components. The architecture is built upon a clear and simple set of concepts: a graph of nodes and edges, a sophisticated system for managing the lifecycle of resources, and a strict contract for declaring dependencies. Together, these elements form a powerful abstraction that separates rendering logic from the complexities of low-level resource management.

2.1 The Anatomy of a Graph: Nodes and Edges

Visually and conceptually, a render graph is composed of boxes (nodes) and arrows (edges) that describe the flow of data through the rendering pipeline for a single frame.¹³

Execution Nodes

Execution nodes are the vertices of the graph and represent a discrete unit of work to be performed by the GPU.¹³ These nodes are designed to be self-contained and modular, encapsulating a specific piece of rendering logic.⁷ The primary types of execution nodes are:

- **Render Passes:** This is the most prevalent type of node. A render pass typically consists of a sequence of draw calls that render geometry to a set of output textures (render targets).¹³ Examples include a G-buffer pass in a deferred renderer, a shadow map generation pass, or a post-processing pass that applies a bloom effect.
- **Compute Passes:** These nodes execute a compute shader, which is a program designed for general-purpose computation on the GPU (GPGPU).⁶ Compute passes are used for a wide range of tasks, such as light culling, particle simulation, image processing, or accelerating culling for GPU-driven pipelines.
- **Transfer/Blit Nodes:** These nodes represent pure data movement operations, such as copying data from one buffer to another or blitting (a block image transfer) from one texture to another, potentially with format conversion or scaling.¹³

Resource Nodes

Resource nodes represent the data that is produced and consumed by the execution nodes.¹³ They are the inputs and outputs that define the relationships and dependencies between passes. Common resource types include:

- **Textures and Render Buffers:** These are multi-dimensional arrays of data, typically representing images.¹³ This includes color buffers, depth/stencil buffers, G-buffers in deferred rendering, shadow maps, and any intermediate textures used during post-processing chains.⁹
- **Buffers:** These are linear, one-dimensional blocks of memory used for various purposes, such as storing vertex and index data for geometry, uniform data (shader constants), or structured data for compute shaders (storage buffers).⁶

Edges

Edges are the directed arrows that connect the nodes, illustrating the flow of data and thereby establishing the dependency structure of the entire frame.¹³ An edge originating from a resource node and pointing to an execution node signifies that the pass *reads* from that resource (an input dependency). Conversely, an edge originating from an execution node and pointing to a resource node signifies that the pass *writes* to that resource (an output). This explicit graph of dependencies is the foundational information upon which all of the render graph's optimizations are built.

2.2 Resource Management: Virtual, Transient, and External

One of the most powerful aspects of the render graph architecture is its sophisticated approach to resource management. It achieves this by abstracting the concept of a resource, separating its logical representation from its physical memory allocation. This separation is the key to its efficiency.

Virtual Resources and Handles

During the initial setup phase of the graph, the application code does not work with concrete, physical GPU resources. Instead, it operates on lightweight, virtual resource handles.⁶ These handles are essentially logical identifiers—often just an index and a version number—that act as a promise for a resource that will exist at some point in the future.⁶ All render graph APIs use these handles to declare dependencies.¹⁵ The physical GPU memory (a `VkImage` or `ID3D12Resource`) is only allocated and bound to the handle by the render graph system immediately before the first pass that needs to use it is executed.¹⁵ This "just-in-time" physicalization of resources is a core principle.

This use of virtual handles creates a profound and powerful decoupling of rendering logic from memory management logic. A developer writing a shader or a render pass only needs to declare their intent, for example, "this pass writes to a texture with format `R16G16B16A16_FLOAT` and the same dimensions as the main viewport." They do not need to be concerned with the low-level details of how or when that texture's memory is allocated, pooled, or released. The render graph system takes on this responsibility, freeing the developer to focus purely on implementing the feature's logic. This separation of concerns dramatically improves code clarity, reduces the potential for memory management bugs, and is a cornerstone of the system's maintainability.

Transient Resources

Many resources in a rendering pipeline have a very short lifetime, existing only within the confines of a single frame. A classic example is the G-buffer in a deferred renderer: it is created at the beginning of the frame, written to by the geometry pass, read by the lighting pass, and then is no longer needed.⁹ Such resources are known as **transient resources**.⁸

The render graph system has full authority over the lifecycle of transient resources. By analyzing the dependency graph, the system can determine the exact point a resource is first used and the exact point it is last used. It can then allocate physical memory for the resource just before its first use and release that memory immediately after its last use, making it available for other resources.⁹ This dynamic, fine-grained lifetime management ensures that

the application's memory footprint is minimized at any given point during the frame, as memory is only committed for resources that are actively in use.

External/Imported Resources

Not all resources are transient. Some must persist across frame boundaries. The most obvious example is the final image that is presented to the screen (the swapchain or backbuffer). Other examples include textures used for temporal effects like temporal anti-aliasing (TAA) or motion blur, where data from the previous frame is required to render the current one.²⁶ These persistent resources are created and managed outside the render graph's lifecycle. They are then "imported" into the graph for a given frame.⁶ Once imported, they behave like any other resource within the context of that single frame; the graph tracks their usage to resolve dependencies and insert correct barriers. However, the graph does not control their creation or destruction, ensuring their persistence between frames.¹⁵

2.3 Dependency Declaration and Resource Versioning

The entire render graph system is predicated on a simple but strict contract: every pass must explicitly and completely declare its resource usage.⁶ This explicit declaration is the mechanism by which the dependency graph is constructed.

Explicit Declaration

For each pass added to the graph, the developer must specify two sets of resources:

1. **Inputs:** A list of all resource handles that the pass will read from.
2. **Outputs:** A list of all resource handles that the pass will write to.

A resource can be both an input and an output, signifying a read-modify-write operation (e.g., blending into an existing render target).⁸ This explicit list of reads and writes is the "source code" that the render graph "compiler" uses to understand the data flow of the frame.

Resource Versioning

A common scenario in rendering is for a single logical resource to be modified sequentially by multiple passes. For instance, a full-screen post-processing effect might be a texture that is first written to by a tone-mapping pass, then read and written to by a bloom pass, and finally read and written to by a sharpening pass. To manage these sequential dependencies on the same underlying data, the render graph employs a versioning system.⁶

When a resource is first created, its handle is at version 0. When a pass writes to that

resource, it consumes the current version as an input and produces a new handle with an incremented version number as an output.⁶ A subsequent pass that needs to operate on the result will declare its dependency on this new, higher version. This versioning scheme implicitly and unambiguously defines the required execution order. For example:

1. ToneMappingPass reads SceneColor:v0 and writes PostProcessTex:v0.
2. BloomPass reads PostProcessTex:v0 and writes PostProcessTex:v1.
3. SharpenPass reads PostProcessTex:v1 and writes PostProcessTex:v2.

This chain of dependencies makes it clear to the system that the passes must execute in the order: ToneMapping, then Bloom, then Sharpen.

Section 3: The Execution Pipeline: From Declaration to GPU Submission

The power of the render graph lies not just in its structure, but in its dynamic, multi-phase execution model. Each frame, the graph is built, analyzed, optimized, and finally executed. This process transforms the developer's high-level, declarative intent into an efficient, low-level sequence of GPU commands. Understanding this three-phase pipeline is key to understanding how the render graph achieves its performance and maintainability goals.

3.1 A Three-Phase Process: Setup, Compile, Execute

The render graph's lifecycle is transient; it is built from scratch, optimized, and executed for every single frame.⁹ This is necessary because the set of required rendering passes can change dynamically based on camera position, game state, or user settings. The process is universally broken down into three distinct phases:

1. **Setup Phase:** In this initial phase, the application code defines the structure of the graph. As the engine's main loop determines which features are active for the current frame (e.g., are shadows enabled? is a particular post-effect active?), it makes calls to the render graph API to declare the corresponding passes and specify their resource inputs and outputs using virtual resource handles.⁹ This builds up a raw, unoptimized graph representing all potential work for the frame.
2. **Compilation Phase:** Once all passes have been declared, the compilation phase begins. This is a CPU-side process where the render graph system analyzes the declared graph structure to perform a series of powerful optimizations.⁹ It determines the optimal execution order, calculates precise resource lifetimes, identifies opportunities for memory savings, computes necessary synchronization points, and eliminates any work that does not contribute to the final output. This phase is where the "intelligence" of the system resides.
3. **Execution Phase:** In the final phase, the system traverses the compiled, optimized list

of passes and translates them into concrete GPU commands. It records these commands into one or more command buffers.⁶ Before executing each pass's logic, the system inserts the pre-computed barriers and resource state transitions, allocates physical memory for any newly required transient resources, and binds them. After a pass's last use of a transient resource, the system releases its memory.¹⁵ Finally, the populated command buffers are submitted to the GPU's command queue for execution.

3.2 The Compilation Phase: The Source of Optimization

The compilation phase is the heart of the render graph architecture. The global, frame-wide knowledge captured during the setup phase enables a set of optimizations that would be impossible or impractical to perform in a traditional, immediate-mode renderer. This phase can be thought of as a domain-specific compiler for rendering, translating a high-level description of work into an optimized, low-level execution plan.

Pass Culling (Dead Code Elimination)

The most straightforward optimization is pass culling, which is analogous to dead code elimination in a traditional compiler.⁶ The system begins at the final resource that will be presented to the user (the "root" of the graph, typically the swapchain) and performs a reverse traversal of the dependency graph, marking every pass it visits as "alive".⁶ Any pass that is not visited during this traversal is one whose output is never consumed by any part of the pipeline that contributes to the final image. These un-referenced passes are "dead" and are simply culled, or removed, from the execution list.⁶ This feature is immensely valuable for managing optional or debug-related rendering passes. A developer can unconditionally declare a pass that visualizes, for example, shadow cascades or motion vectors. If that visualization is not ultimately connected to the final frame buffer, the render graph will automatically and safely remove it and all its inputs, incurring zero CPU or GPU cost.¹⁵ This eliminates the need for complex and fragile if/else logic scattered throughout the rendering code.

Topological Sorting

To convert the graph structure into a linear sequence of passes for execution, the system performs a topological sort.⁶ This standard graph algorithm produces an ordering of nodes such that for every directed edge from node A to node B, node A comes before node B in the ordering. In the context of a render graph, this guarantees that no pass is executed before the passes that produce its inputs have completed. The algorithm typically works by finding nodes with no incoming dependencies, adding them to the execution list, removing them and

their outgoing edges from the graph, and repeating the process until the graph is empty. If at any point no node with zero incoming dependencies can be found, it means the graph contains a cycle (e.g., Pass A depends on Pass B, which in turn depends on Pass A). This is an invalid state, representing a logical bug in the renderer's setup, which the system can detect and report as an error.⁶

Resource Lifetime Analysis and Memory Aliasing

During compilation, the system analyzes the now-linearized execution list to determine the precise lifetime of every transient resource: the interval between its first use (typically a write) and its last use (the final read).⁴ With these lifetimes known, the system can perform a memory optimization known as

aliasing.⁴ If the lifetime of transient resource A does not overlap with the lifetime of transient resource B, and they have compatible memory requirements (e.g., size and format), the system can assign both virtual resources to the same underlying block of physical GPU memory.²⁶ For example, the memory used for the G-buffer can be reused later in the frame for a full-screen bloom texture after the lighting pass has finished reading from it. This technique dramatically reduces the peak GPU memory consumption of a frame without requiring any manual memory management from the feature developer.⁹

Barrier Computation

With a complete understanding of the resource dependencies and the intended usage (read or write) in each pass, the system can automatically compute and insert the minimal set of synchronization primitives required for correctness.⁷ It knows exactly when a resource needs to be transitioned from a writeable state (e.g., `COLOR_ATTACHMENT_OPTIMAL`) to a readable state (e.g., `SHADER_READ_ONLY_OPTIMAL`) and can inject the appropriate pipeline barrier at the correct point in the command buffer.⁸ This automates one of the most complex and error-prone aspects of modern graphics programming, preventing a wide class of bugs like race conditions and visual corruption, while ensuring that synchronization is no more conservative than necessary, thus avoiding unnecessary GPU pipeline stalls.⁴

3.3 Retained Mode vs. Immediate Mode: An Architectural Comparison

The render graph's deferred execution model firmly places it in the category of a **retained-mode** graphics API architecture.¹ In this model, the application builds up a description of the scene—in this case, the graph of passes and resources—which is then "retained" by the system. The system, not the application, is then responsible for traversing

this data structure and issuing the low-level drawing commands.¹

This stands in stark contrast to the **immediate-mode** paradigm of older APIs, where the application's API calls are (conceptually) translated directly and immediately into drawing commands.¹ The graphics library in an immediate-mode system does not store a high-level model of the scene between frames; that responsibility lies with the application.²⁷

The primary advantage of the retained-mode approach is its capacity for powerful, holistic optimization. Because the system can see the entire frame's workload before any commands are sent to the GPU, it can perform the complex analysis and optimizations described above.⁵ Immediate mode, while often simpler to reason about procedurally, limits the system to more localized optimizations, as it has no knowledge of what commands may be coming next. The render graph architecture, therefore, represents a deliberate choice to trade the procedural simplicity of immediate mode for the global optimization power of retained mode, a trade-off that is essential for achieving peak performance on modern hardware.

Feature	Immediate Mode (e.g., raw DX11/OpenGL)	Retained Mode (Render Graph)
Paradigm	Procedural: "Do this, then do that."	Declarative: "This is what I want to render."
Execution	Commands executed immediately as they are issued.	Execution is deferred until the entire frame is declared.
State Mgt.	Application manually tracks and transitions state.	System manages state transitions based on declarations.
Resource Mgt.	Manual allocation/deallocation; worst-case memory planning.	Automated lifetime management; transient resources and aliasing.
Synchronization	Managed by driver (implicit) or manually placed barriers.	Automated, optimal barrier placement by the system.
Optimization	Local, per-command optimization. Difficult to reason globally.	Global, whole-frame optimization (pass culling, merging, reordering).
Flexibility	High flexibility, but high complexity.	Less direct control, but abstracts away complexity.
Developer Focus	Low-level command submission and synchronization.	High-level feature logic and pass dependencies.

Section 4: The Benefits of a Graph-Based Architecture

The adoption of the render graph architecture by virtually every major real-time rendering engine is a testament to its profound benefits. These advantages are not merely incremental performance tweaks; they represent a fundamental improvement in how rendering pipelines are designed, implemented, and maintained. The benefits can be broadly categorized into two main areas: direct performance gains through superior GPU and memory optimization, and significant improvements in the development process itself, leading to more robust, modular, and maintainable code.

4.1 Performance Gains: Optimizing GPU and Memory Usage

The primary driver for the creation of render graphs was the pursuit of performance. By providing a holistic, frame-wide view of the rendering workload, the architecture enables a class of optimizations that are difficult or impossible to achieve with traditional, immediate-mode approaches.

Efficient Memory Management

One of the most significant performance benefits is a drastic reduction in GPU memory usage.¹⁰ In a traditional renderer, resources often must be allocated based on a worst-case scenario, accounting for a situation where every possible rendering feature is active simultaneously.¹⁰ This leads to a large, static memory footprint, much of which may go unused in a typical frame.

The render graph completely upends this model. Through its management of **transient resources**, the system only allocates physical memory for resources that are actually used in the current frame.¹⁰ Furthermore, through **resource aliasing**, it reuses the same slabs of physical memory for different transient resources whose lifetimes do not overlap.⁴ This "just-in-time" allocation and aggressive reuse strategy ensures that the peak memory footprint of the renderer is as close as possible to the actual working set of data for that specific frame. This not only reduces overall memory pressure but also frees up valuable VRAM that can be used to implement more ambitious visual effects.¹⁰

Automatic Synchronization and Reduced Stalls

Manually managing synchronization in explicit APIs is a major source of performance loss. Developers, wary of introducing subtle race conditions, often resort to overly conservative, "big hammer" barriers that stall the GPU pipeline unnecessarily. The render graph solves this by automating the entire process.¹⁰

Because the graph has complete knowledge of how and when each resource is read from and

written to, it can calculate and insert the most precise and minimally restrictive barriers, fences, and semaphores required for correctness.⁴ It can schedule layout transitions to occur as early as possible, hiding their latency behind other work and preventing pipeline bubbles.⁴ This is especially critical for complex scenarios involving **asynchronous compute**, where work on the compute queue runs in parallel with the graphics queue. The render graph can automatically generate the correct synchronization points (semaphores) to manage dependencies between these queues, a task that is notoriously difficult to perform correctly and efficiently by hand.¹⁰

Pass Merging and Elimination

The graph's high-level representation allows it to restructure the workload for better hardware utilization. The pass culling mechanism directly reduces the total amount of work sent to the GPU by eliminating any rendering that does not contribute to the final image.⁶ Moreover, the system can identify sequences of passes that can be merged into a single, more efficient operation. This is particularly impactful on tile-based rendering architectures (TBDR), common in mobile and integrated GPUs. On these architectures, merging multiple passes that operate on the same render targets into a single Vulkan `VkRenderPass` with multiple subpasses allows intermediate results to stay in extremely fast on-chip tile memory, avoiding slow and power-hungry round trips to external DRAM.⁸ This can result in significant performance and power-efficiency gains.

4.2 Development and Maintainability: A Cleaner, More Robust Approach

Beyond raw performance, the render graph architecture provides transformative benefits for the software engineering process. It imposes a structure that leads to cleaner, more modular, and more resilient codebases.

Modularity and Reusability

The architecture naturally encourages developers to encapsulate rendering features into self-contained, modular passes with explicitly declared inputs and outputs.⁷ A bloom filter, for example, can be implemented as a series of passes that declare an input texture and produce an output texture. This module can then be easily "plugged in" anywhere in the rendering pipeline where that contract can be satisfied, without needing to modify the surrounding code.⁷ This high degree of modularity makes features easier to reuse in different contexts, simplifies unit testing, and allows for the creation of a library of rendering effects that can be composed in flexible ways. It avoids the large amounts of duplicated code and complex

conditional logic that plague monolithic renderers.⁷

This inherent modularity and safety also lowers the barrier to architectural experimentation. In a traditional renderer, inserting a new pass or swapping rendering techniques can be a perilous undertaking, requiring careful manual intervention at multiple points in the code to manage resources and synchronization. The render graph provides a safety net. A developer can confidently "patch" a new feature, such as ray-traced reflections, into an existing graph.⁷ The system guarantees that if the dependencies are correctly declared, the new pass will be integrated with the correct barriers and resource transitions. This agility transforms the renderer from a rigid, brittle system into a flexible, composable one, accelerating research and development and allowing teams to iterate on new visual ideas more rapidly.

Simplified Code and Reduced Complexity

Resource management is consistently cited as one of the most difficult aspects of maintaining a large rendering pipeline.¹⁰ By abstracting away the details of resource allocation, lifetime tracking, state transitions, and synchronization, the render graph allows developers to focus on the actual rendering logic of their feature.⁹ The resulting feature code is simpler, cleaner, and more focused on its specific task. This reduction in cognitive overhead not only speeds up development but also reduces the surface area for bugs, leading to a more robust and stable renderer.

Improved Validation and Debugging

Because the entire frame is declared before it is executed, the render graph system is in a unique position to perform powerful, upfront validation.⁴ During the setup or compilation phase, it can automatically detect a range of common functional and performance-related issues, such as circular dependencies, invalid resource states, or attempts to use a resource in an incompatible way.⁴ Catching these errors early in the development process is far more efficient than debugging subtle graphical corruption on the GPU.

Furthermore, the graph structure itself is a powerful debugging tool. Visualization tools can consume the graph data and present an interactive diagram of the entire frame's data flow.²⁴ This allows developers to visually inspect resource lifetimes, identify which passes are being culled or merged, and trace the path of a resource through the pipeline, making it vastly easier to understand and debug the behavior of a complex, multi-pass renderer.¹²

Section 5: Implementation Deep Dive: Render Graphs and Modern Graphics APIs

The high-level concepts of a render graph—passes, resources, and dependencies—must ultimately be translated into the concrete, low-level primitives of a specific graphics API. While the core principles remain the same, the implementation details differ significantly between APIs like Vulkan and DirectX 12, each offering unique features and mechanisms for synchronization and state management. The render graph serves as a crucial abstraction layer, providing a consistent programming model that can be compiled down to the most efficient primitives for the target platform. The evolution of this concept is now pushing towards even greater GPU autonomy with features like Work Graphs.

5.1 Vulkan Implementation: Leveraging Explicit Control

Vulkan's explicit nature and rich feature set provide a powerful foundation for implementing a render graph. Its design, particularly around render passes and synchronization, maps very naturally to the graph's structure.

Mapping Passes to `VkRenderPass`

A key feature of Vulkan, especially for tile-based architectures common on mobile devices, is the `VkRenderPass` object. A single `VkRenderPass` can be configured with multiple **subpasses**. These subpasses execute sequentially, and crucially, the results of one subpass can be passed directly to the next as an input attachment without ever leaving the GPU's fast on-chip memory.⁸ This avoids costly round-trips to external DRAM, saving significant memory bandwidth and power.²⁴

A sophisticated render graph implementation can analyze the graph for sequential chains of passes that operate on the same render targets and automatically merge them into a single `VkRenderPass` with multiple `VkSubpassDependency` structures to manage the data flow between them.⁸ For example, a G-buffer pass followed immediately by a lighting pass that consumes the G-buffer as input attachments is a prime candidate for merging.

Managing Dependencies with `VkSubpassDependency` and Barriers

The render graph's compilation phase is responsible for translating the logical dependencies between passes into concrete Vulkan synchronization primitives.

- **Intra-Pass Synchronization:** For dependencies between subpasses within a merged `VkRenderPass`, the system generates `VkSubpassDependency` structures. These structures precisely define the memory and execution dependencies, specifying which pipeline stages in a subsequent subpass must wait for which stages in a previous subpass to complete.⁸
- **Inter-Pass Synchronization:** For dependencies between separate `VkRenderPass`

objects or between graphics and compute passes, the system emits a `vkCmdPipelineBarrier` command. This is a more general-purpose barrier that can synchronize access to resources (buffers and images) and perform image layout transitions.⁸ The graph's global knowledge ensures these barriers are as fine-grained as possible, specifying the narrowest required pipeline stages (`srcStageMask`, `dstStageMask`) and access scopes (`srcAccessMask`, `dstAccessMask`) to maximize pipeline parallelism.

Synchronization Primitives

For more complex synchronization, particularly between different hardware queues (e.g., graphics and async compute), Vulkan provides more powerful primitives that a render graph can leverage:

- **VkEvent:** Events are fine-grained synchronization primitives used for dependencies *within a single queue*. A pass can signal an event at a specific pipeline stage, and a later pass can wait on that event. This allows for more overlap in execution than a full pipeline barrier.⁸ The render graph can track the last event to affect a resource to manage these dependencies.
- **VkSemaphore:** Semaphores are used to synchronize work *between different queues*. For example, a compute pass on the compute queue can signal a semaphore upon completion, and the graphics queue can be made to wait for that semaphore before a graphics pass that consumes the compute pass's results can begin. The render graph automates this by tracking cross-queue resource usage and managing the submission of wait and signal semaphore operations.⁸

5.2 DirectX 12 Implementation: State Transitions and Fences

DirectX 12 shares Vulkan's explicit philosophy but uses a different set of primitives to manage state and synchronization. A render graph implementation targeting DX12 focuses on managing these specific mechanisms.

Resource State Transitions

A core concept in DirectX 12 is that every resource exists in a specific state at any given time (e.g., `D3D12_RESOURCE_STATE_RENDER_TARGET`, `D3D12_RESOURCE_STATE_PIXEL_SHADER_RESOURCE`, `D3D12_RESOURCE_STATE_COPY_DEST`). Before a resource can be used in a particular way, it must be transitioned to the correct state. This is done by recording a `D3D12_RESOURCE_BARRIER` of type `TRANSITION` into a command list.⁴

Manual management of these state transitions is a major source of complexity and bugs in DX12 development. The render graph completely automates this process. During compilation, it tracks the required state of each resource for every pass it is used in. It then computes the minimal set of transition barriers needed and inserts them at the appropriate points during the execution phase.⁴

Aliasing Barriers

The render graph's ability to alias transient resources onto the same physical memory is mapped directly to a specific DX12 feature. When the system reuses a piece of memory for a new resource, it emits a D3D12_RESOURCE_BARRIER of type ALIASING. This barrier informs the driver and hardware that the contents of the memory are now associated with a different resource, resolving potential data hazards.⁹

Cross-Queue Synchronization

Similar to Vulkan's semaphores, DirectX 12 uses **Fences** (ID3D12Fence) for synchronizing work across different command queues (Graphics, Compute, and Copy).⁴ To create a dependency, the producing queue signals a fence to a specific value upon completion of its work. The consuming queue then issues a command to wait until the fence reaches that value before executing its dependent work. The render graph automates this entire process, tracking cross-queue dependencies, inserting the Signal and Wait commands, and managing the fence values transparently.

5.3 The Next Evolution: GPU-Driven Pipelines and Work Graphs

The render graph architecture, while powerful, is fundamentally a CPU-side scheduling system. It determines the frame's structure on the CPU and then submits a largely static set of commands to the GPU. The next major evolution in rendering architecture pushes even more of this scheduling and work generation directly onto the GPU itself, a paradigm known as **GPU-driven rendering**.³¹

DirectX 12 Work Graphs

Work Graphs are a new, first-class feature in DirectX 12 (and a corresponding extension is in development for Vulkan) that formalizes and extends the concepts of GPU-driven pipelines.³¹ A Work Graph is a graph of compute-style shaders, called nodes, that can dynamically launch work on other nodes, all without any CPU intervention.³³ This creates a powerful

producer-consumer model that lives entirely on the GPU. For example, a culling node could process a list of objects and, for each visible object, launch a specific material-shading node to process it. This is a more flexible and powerful replacement for older techniques like `ExecuteIndirect`, which were more constrained.³¹

Mesh Nodes

A recent and significant enhancement to Work Graphs is the introduction of **mesh nodes**.³¹ These are a special type of leaf node in the graph that can drive a mesh shader. This finally closes the loop, allowing a fully GPU-driven compute pipeline (the Work Graph) to dynamically generate and dispatch work for the graphics rasterization pipeline. For example, a Work Graph could perform procedural geometry generation, culling, and level-of-detail (LOD) selection entirely on the GPU, and then launch mesh shader nodes to render the final, processed geometry. This represents a major step towards a future where the CPU's role is simply to kick off a high-level graph, with the GPU autonomously managing the entire rendering process from scene traversal to final pixels.³¹

Section 6: Case Studies: Render Graphs in the Wild

The theoretical principles of render graphs have been validated through their successful implementation in some of the world's most advanced and widely used real-time rendering engines. Examining these real-world case studies reveals how the core concepts are adapted to fit different engine philosophies, APIs, and developer ecosystems. Each implementation, while sharing the same foundational goals, possesses unique terminology, features, and tooling that reflect its specific context.

6.1 Unreal Engine's Rendering Dependency Graph (RDG)

Unreal Engine's implementation is known as the **Rendering Dependency Graph (RDG)**. It is a sophisticated, graph-based scheduling system deeply integrated into the engine's rendering architecture, designed specifically for whole-frame optimization on modern, explicit APIs.⁴

Architecture

The RDG operates on the core render graph principle of deferred execution, employing a retained-mode model where the entire frame's rendering passes are recorded into a dependency graph data structure before any GPU commands are executed.⁴ This graph is built and executed on Unreal's dedicated "Render Thread," which communicates with the

lower-level Render Hardware Interface (RHI) layer.³⁷ The RDG acts as a high-level abstraction on top of RHI command lists, providing a more structured and optimized way to generate GPU work.³⁷

Key Features

The RDG is engineered to automatically handle the most complex aspects of modern API programming:

- **Automated Scheduling:** It performs automatic scheduling and fencing of asynchronous compute passes, maximizing parallelism between the graphics and compute pipelines.⁴
- **Efficient Memory Management:** The RDG has a highly optimized memory manager that performs automatic resource aliasing. It analyzes the lifetimes of transient resources (like render targets and buffers created within the graph) and maps them to overlapping regions of pooled physical memory, significantly reducing the frame's memory footprint.⁴
- **Optimized Synchronization:** It intelligently kicks off resource barriers and layout transitions as early as possible in the frame to hide their latency and avoid GPU pipeline stalls.⁴
- **Validation:** A significant advantage of the RDG is its rich validation layer. During the pass setup phase, it can automatically detect and report a wide range of functional and performance-related errors, such as incorrect resource flags or missing dependencies, providing clear feedback to developers and improving stability.⁴
- **Cinematic Rendering:** The RDG's principles are also extended to Unreal's high-quality offline rendering toolset through the **Movie Render Graph**, which allows for the construction of complex, layered rendering pipelines for producing cinematic-quality output.³⁸

6.2 Unity's Render Graph System

Unity introduced its **Render Graph** system as a core component of its modern, customizable **Scriptable Render Pipeline (SRP)** architecture. It serves as the foundation for both the Universal Render Pipeline (URP) and the High Definition Render Pipeline (HDRP).¹¹

Context

The primary goal of Unity's Render Graph is to provide a maintainable and modular way to author a custom render pipeline.¹¹ It offers a formal API that sits on top of the lower-level SRP commands, abstracting away the complexities of resource and state management.

Developers interact with the system to create custom

Renderer Features, which are self-contained modules that can be used to inject custom rendering logic (like post-processing effects or custom drawing passes) into the main pipeline.²⁸

API and Usage

Developers use the RenderGraph API to add passes to the graph. The system manages specific resource types, including RTHandles (Unity's render target handle system), ComputeBuffers, and RendererLists (a mechanism for drawing lists of objects).¹⁵ When adding a pass, the developer defines a PassData structure to hold any parameters and resource handles the pass needs. The actual execution logic is provided as a delegate or lambda function that receives this data during the execution phase.⁴² This design enforces a clean separation between the declaration of a pass and its implementation.

Developer Tooling

A key strength of Unity's implementation is its excellent developer tooling. Unity provides the **Render Graph Viewer**, a powerful analysis window that gives developers a visual representation of the final, compiled graph for a frame.²⁸ This viewer allows developers to:

- Inspect the full sequence of executed passes.
- Visualize the lifetimes of transient resources, seeing exactly when they are created and destroyed.
- Identify which passes have been merged by the system for optimization.
- Analyze resource dependencies and data flow through the entire frame.

This tool is invaluable for debugging rendering issues and optimizing performance, as it provides a clear and accurate picture of what the renderer is actually doing on the GPU.³⁰

6.3 Other Notable Implementations

While Unreal and Unity are prominent examples, the render graph pattern is widespread.

Frostbite's FrameGraph

As the system that popularized the architecture, Frostbite's **FrameGraph** was designed from the ground up to address the challenges of supporting a wide variety of vastly different games (from first-person shooters to sports titles) on a single, unified engine.¹⁶ Its primary

architectural goals were to enable the implementation of rendering features in a decoupled, modular way, allowing features to be developed independently and plugged into the engine without causing cascading changes, all while maintaining the high level of efficiency required for AAA games.¹⁶

Babylon.js Frame Graph

The render graph pattern is not limited to native C++ engines. The popular web-based 3D engine **Babylon.js** also features a **Frame Graph** implementation.⁴³ In the context of WebGL/WebGPU, it serves the same core purposes: simplifying and optimizing resource management, and providing a more flexible and powerful alternative to the engine's older, hard-coded rendering pipeline. It allows developers to define custom rendering flows by creating tasks and their interconnections in a graph structure.⁴³

Ogre3D Compositor

The open-source engine **Ogre3D** features a system called the **Compositor**, which, in its modern incarnation, embodies the principles of a render graph.⁴⁴ The Ogre 2.0+ Compositor is a node-based system where the developer defines a "Workspace" by connecting various nodes (such as `render_scene`, `quad`, `clear`). This graph defines the entire rendering pipeline, managing render targets, input/output channels between nodes, and the sequence of effects. It is a powerful, data-driven system that replaces the older, more limited stack-based post-processing framework of previous versions, demonstrating the convergence of rendering architectures around graph-based designs.⁴⁴

The evolution of these systems points toward a broader trend. While early implementations like Frostbite's were primarily code-driven, with developers writing C++ callbacks to define pass logic, the paradigm is increasingly shifting towards a more data-oriented approach. Systems like the Cocos engine's Render Graph and Unreal's Movie Render Graph treat the graph itself as a serializable data asset.¹⁴ This allows the rendering pipeline to be defined and modified not just in code, but through visual editors and data files. This empowers technical artists and other developers to customize rendering without deep C++ knowledge and enables more dynamic and flexible pipeline configurations.

Feature	Frostbite FrameGraph (c. 2017)	Unreal Engine RDG	Unity Render Graph
Primary Goal	Modular, extensible architecture for a multi-game engine.	Whole-frame optimization for modern APIs.	Maintainable and modular SRP authoring.
Core Abstraction	Graph of render passes and resources.	Dependency graph for deferred execution.	API for creating a Scriptable Render

			Pass.
Resource Handling	Transient resource system, resource aliasing.	Automatic memory aliasing, pooling.	Manages RTHandles, ComputeBuffers.
Async Compute	A key design goal for simplification.	Automatic scheduling and fencing.	Automatic synchronization point generation.
Developer API	Callback/lambda-based pass execution.	FRDGBuilder API, pass parameter structs.	RenderGraph.AddRasterRenderPass, PassData.
Tooling	Internal visualization tools.	In-engine profilers, VisLog.	Render Graph Viewer, Frame Debugger.

Section 7: Practical Application: Debugging, Optimization, and Best Practices

Working effectively with a render graph requires a shift in mindset from traditional, procedural rendering. Because the system abstracts away low-level details and may reorder or modify the workload, developers need a new set of tools and best practices to debug, optimize, and author rendering features. The key is to understand the graph's optimization strategies and to write code that enables, rather than hinders, them.

7.1 Visualizing the Frame: Understanding the Execution Flow

In an immediate-mode renderer, the execution flow directly mirrors the code's structure. In a render graph, this is no longer true; passes may be culled, merged, or reordered. Therefore, visual debugging tools are not just a convenience—they are essential for understanding what the GPU is actually doing.¹²

Tools of the Trade

A variety of powerful tools exist to help developers inspect and analyze the render graph:

- **Engine-Integrated Viewers:** Many engines provide built-in tools. Unity's **Render Graph Viewer** is a prime example, offering a clear visualization of the final compiled graph, including pass merging and the precise lifetimes of transient resources.²⁸ These tools are the first line of defense for understanding the high-level structure of the

frame.

- **GPU Performance Analyzers:** Tools like Arm's **Frame Advisor** can capture a frame and generate a render graph visualization that shows the relationships between workloads and resources.¹² These tools are excellent for identifying performance issues like unnecessary memory reads and writes between passes, which indicate opportunities for pass merging.²⁴
- **Frame Debuggers:** Low-level frame debuggers like **RenderDoc**, PIX for Windows, and Xcode's Metal debugger allow for step-by-step inspection of every draw call and API command in a frame.⁴⁷ When a render graph is active, these tools will typically show a high-level event for the entire graph execution, with child events corresponding to the individual compiled passes.³⁰ They are indispensable for debugging low-level graphical artifacts or validating the barriers and state transitions generated by the graph.

7.2 Common Pitfalls and Anti-Patterns

While render graphs solve many problems, they also introduce new potential pitfalls. Many of these arise from a misunderstanding of the graph's declarative nature and can subvert its optimization capabilities. These anti-patterns are often the inverse of the graph's primary benefits; they are design choices that prevent the system's automated optimizations from working effectively.

Circular Dependencies

This is the most fundamental logical error in a graph-based system. It occurs when a cycle is introduced into the dependency graph—for example, Pass A is declared to read the output of Pass B, but Pass B is also declared to read the output of Pass A. This creates a logical impossibility that cannot be resolved into a linear execution order. A correctly implemented render graph will detect this during the topological sort phase of compilation and raise a critical error, as it indicates a flaw in the renderer's logic.⁶

Inefficient Resource Barriers and False Dependencies

The graph's ability to generate optimal barriers is one of its greatest strengths. A common pitfall is to introduce "false dependencies" that force the system to generate overly conservative barriers, creating unnecessary GPU stalls. For example, declaring that a pass reads an entire G-buffer when it only needs to sample the depth component can create a dependency that prevents that pass from running in parallel with another pass that is writing to the color components of the G-buffer. Developers must be precise in declaring only the resources that are truly required for a pass to function.

"Phyllo Graphs" and Excessive Pass Granularity

This anti-pattern, named by analogy to data visualization pitfalls, describes the practice of breaking down the rendering pipeline into an excessive number of very small, fine-grained passes.⁵¹ While modularity is a key benefit of the render graph, extreme granularity can be counterproductive. Each pass incurs a small amount of CPU overhead for setup, compilation, and execution. A graph with thousands of tiny passes can lead to significant CPU-side overhead that outweighs the GPU benefits. Furthermore, such a graph becomes nearly impossible to visualize and debug effectively, resembling a tangled mess of nodes.⁵¹ A good design finds a balance, grouping related operations into reasonably sized passes.

Mismanaging Persistent Resources

A common mistake is to forget the distinction between transient and external resources. The render graph does *not* manage the lifetime of resources imported from outside the graph.¹⁵ Developers are still responsible for creating, destroying, and managing the state of these persistent resources between frames. Assuming the graph will handle a resource that was manually created can lead to resource leaks or using a resource that contains stale data from a previous frame.

7.3 Optimization Strategies and Best Practices

Writing high-performance code for a render graph system involves learning to "think like the graph compiler." The goal is to structure the declaration of passes and resources in a way that exposes the maximum number of optimization opportunities to the system.

Maximize Async Compute

Identify computationally intensive workloads that do not have direct dependencies on the main rasterization pipeline (e.g., particle simulations, some physics calculations, procedural generation). By declaring these as compute passes with minimal dependencies on the graphics passes, the developer signals to the render graph scheduler that they are excellent candidates to be run in parallel on the asynchronous compute queue, maximizing overall GPU occupancy.

Optimize for Tiled Architectures

When developing for mobile platforms, which predominantly use tile-based rendering architectures, it is crucial to minimize memory bandwidth. This can be achieved by structuring passes that operate on the same set of render targets to be sequential in the graph. This increases the likelihood that the render graph compiler will merge them into a single `VkRenderPass` with multiple subpasses, allowing intermediate data to be passed via on-chip tile memory and avoiding expensive writes and reads to and from main memory.⁸

Minimize Resource Lifetimes

To enable the memory aliasing optimizer to work most effectively, the lifetimes of transient resources should be kept as short as possible. A resource should only be declared as an input to the passes that strictly require it. Artificially keeping a resource "alive" by adding unnecessary dependencies will prevent its underlying memory from being reused, increasing the frame's peak memory footprint.

Leverage Pass Culling for Optional Features

For any rendering that is optional, such as debug visualizations or quality-level-specific effects, declare them as passes that produce a new, unique output resource. Do not modify an existing resource in the main rendering chain. This ensures that if this final output is not connected to the swapchain, the entire chain of passes required to produce it will be safely and automatically culled by the graph with zero runtime cost.¹⁵ This is a far cleaner and more robust solution than using complex conditional logic in the application code.

Section 8: Conclusion and Future Directions

The render graph has firmly established itself as a foundational architectural pattern in modern real-time graphics. It is not merely an optimization technique but a comprehensive solution to the fundamental challenges posed by the shift to explicit, low-level graphics APIs. By providing a declarative, retained-mode abstraction, it successfully re-centralizes the frame-wide knowledge necessary for holistic optimization, while simultaneously promoting a modular, maintainable, and robust software architecture.

8.1 Summary: The Render Graph's Impact on Real-Time Graphics

The analysis presented in this report demonstrates that the render graph architecture delivers a powerful synthesis of performance and productivity. Its core contributions can be summarized as follows:

- **It Tames Complexity:** The render graph automates the most difficult and error-prone aspects of modern graphics programming—namely, resource lifetime management and GPU synchronization. By taking on the responsibility for memory aliasing, state transitions, and barrier placement, it allows developers to focus on the creative and logical aspects of implementing visual effects.
- **It Maximizes Performance:** Through global, whole-frame analysis, the system unlocks a suite of optimizations that are impractical in traditional renderers. Pass culling reduces GPU workload, intelligent barrier placement minimizes pipeline stalls, and transient resource aliasing dramatically lowers peak memory consumption.
- **It Fosters Better Engineering:** The architecture inherently encourages a modular, component-based design. Features become self-contained passes with explicit data dependencies, making them easier to develop, test, reuse, and maintain. This leads to more scalable and resilient rendering codebases.

In essence, the render graph provides a crucial abstraction layer that bridges the gap between high-level rendering intent and the low-level mechanics of the GPU, making the power of modern hardware accessible and manageable for large-scale application development.

8.2 Future Trends: Towards Fully GPU-Driven Pipelines

The architectural philosophy of the render graph—abstracting rendering intent and deferring execution to an intelligent scheduler—points toward a clear future trajectory: moving even more of the scheduling and work generation from the CPU onto the GPU itself. The CPU-side render graph is a powerful scheduler, but it still submits a relatively static workload for a given frame. The next frontier is to empower the GPU to make its own dynamic decisions about what work to perform next.

This future is already taking shape with the introduction of **Work Graphs** in DirectX 12 and their eventual arrival in other APIs.³¹ Work Graphs extend the graph paradigm to run entirely on the GPU. A shader node can analyze data and dynamically launch new work on other nodes, creating complex, data-dependent workflows without any need for a round trip to the CPU.³³

The recent addition of **mesh nodes** represents a critical milestone, allowing a compute-based Work Graph to directly drive the rasterization pipeline.³¹ This enables true end-to-end, GPU-driven rendering systems where, for example, the GPU can perform scene traversal, object and triangle culling, and LOD selection, and then dynamically dispatch mesh shaders to render only the precisely necessary geometry.

This trend represents a continuation of the same principles that made the CPU-side render graph successful: abstracting the "what" from the "how" and giving the underlying system—in this case, the GPU itself—the autonomy to execute that intent in the most efficient manner possible. As this technology matures, we can expect to see rendering pipelines that are more dynamic, more scalable, and even more efficient, continuing the relentless drive for greater

realism and performance in real-time graphics.

Cytowane prace

1. Retained Mode Versus Immediate Mode - Win32 apps | Microsoft ..., otwierano: sierpnia 14, 2025, <https://learn.microsoft.com/en-us/windows/win32/learnwin32/retained-mode-versus-immediate-mode>
2. Should I use retained mode or immediate mode - Game Development Stack Exchange, otwierano: sierpnia 14, 2025, <https://gamedev.stackexchange.com/questions/137543/should-i-use-retained-mode-or-immediate-mode>
3. Any good tutorial about directx 12? : r/GraphicsProgramming - Reddit, otwierano: sierpnia 14, 2025, https://www.reddit.com/r/GraphicsProgramming/comments/1ljtcn/any_good_tutorial_about_directx_12/
4. Rendering Dependency Graph | Unreal Engine 4.27 Documentation ..., otwierano: sierpnia 14, 2025, https://dev.epicgames.com/documentation/en-us/unreal-engine/rendering-dependency-graph?application_version=4.27
5. It's important to point out why games use immediate mode GUIs: 1. The GUI needs - Hacker News, otwierano: sierpnia 14, 2025, <https://news.ycombinator.com/item?id=19768156>
6. Render Graph 101. Designing an efficient renderer can be... | by Manon Oomen | Traverse Research, otwierano: sierpnia 14, 2025, <https://blog.traverseresearch.nl/render-graph-101-f42646255636>
7. The Render Graph Architecture - Adept Engine Dev blog, otwierano: sierpnia 14, 2025, <https://andrewcjp.wordpress.com/2019/09/28/the-render-graph-architecture/>
8. Render graphs and Vulkan — a deep dive – Maister's Graphics Adventures, otwierano: sierpnia 14, 2025, <https://themaister.net/blog/2017/08/15/render-graphs-and-vulkan-a-deep-dive/>
9. Render Graphs - Riccardo Loggini, otwierano: sierpnia 14, 2025, <https://logins.github.io/graphics/2021/05/31/RenderGraphs.html>
10. Benefits of the render graph system | Scriptable Render Pipeline Core - Unity - Manual, otwierano: sierpnia 14, 2025, <https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@17.0/manual/render-graph-benefits.html>
11. The render graph system | Core RP Library | 10.2.2 - Unity - Manual, otwierano: sierpnia 14, 2025, <https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@10.2/manual/render-graph-system.html>
12. What are render graphs? | Arm Learning Paths, otwierano: sierpnia 14, 2025, <https://learn.arm.com/learning-paths/mobile-graphics-and-gaming/render-graph-optimization/what-are-render-graphs/>

13. Understanding your render graph | Arm Learning Paths, otwierano: sierpnia 14, 2025,
<https://learn.arm.com/learning-paths/mobile-graphics-and-gaming/render-graph-optimization/understanding-your-render-graph/>
14. Render Graph - GDC Vault, otwierano: sierpnia 14, 2025,
<https://uat.gdcvault.com/play/mediaProxy.php?sid=1028845>
15. Render graph fundamentals | Core RP Library | 14.0.12 - Unity - Manual, otwierano: sierpnia 14, 2025,
<https://docs.unity3d.com/Packages/com.unity.render-pipelines.core@14.0/manual/render-graph-fundamentals.html>
16. FrameGraph: Extensible Rendering Architecture in Frostbite - GDC Vault, otwierano: sierpnia 14, 2025,
<https://www.gdcvault.com/play/1024612/FrameGraph-Extensible-Rendering-Architecture-in>
17. FrameGraph: Extensible Rendering Architecture in Frostbite - GDC Vault, otwierano: sierpnia 14, 2025,
<https://www.gdcvault.com/play/1024045/FrameGraph-Extensible-Rendering-Architecture-in>
18. Publications - Yuriy O'Donnell, otwierano: sierpnia 14, 2025,
<https://kayru.org/publications/>
19. Scriptable Render Pipeline - Aras Pranckevičius, otwierano: sierpnia 14, 2025,
https://aras-p.info/texts/files/2017_GDC_UnityScriptableRenderPipeline.pdf
20. Designing a modern high-level graphics API, otwierano: sierpnia 14, 2025,
<https://dcgi.fel.cvut.cz/wp-content/wpallimport-dist/theses/pdf/theses-2020-galajrom-thesis.pdf>
21. - Kahns Algorithmus - Visual Computing WH - Westfälische Hochschule, otwierano: sierpnia 14, 2025,
<https://vc.w-hs.de/teaching/gip2-2025/060/040/gip2-060-040-graph-algos-kahn--handout-pre-lecture.pdf>
22. FrameGraph: Extensible Rendering Architecture in Frostbite | PPTX - SlideShare, otwierano: sierpnia 14, 2025,
<https://www.slideshare.net/slideshow/framegraph-extensible-rendering-architecture-in-frostbite/72795495>
23. Creating a render graph for hobby engine? : r/GraphicsProgramming - Reddit, otwierano: sierpnia 14, 2025,
https://www.reddit.com/r/GraphicsProgramming/comments/1lhezg5/creating_a_render_graph_for_hobby_engine/
24. Analyze your frame using the Render Graph - Arm Developer, otwierano: sierpnia 14, 2025,
<https://developer.arm.com/documentation/102693/latest/Analyzing-your-frames/Analyze-your-frame-using-the-Render-Graph>
25. An update to our Render Graph - Traverse Research, otwierano: sierpnia 14, 2025,
<https://blog.traverserresearch.nl/an-update-to-our-render-graph-17ca4154fd23>
26. Render graphs - apoorvaj.io, otwierano: sierpnia 14, 2025,
<https://apoorvaj.io/render-graphs-1/>

27. Immediate mode (computer graphics) - Wikipedia, otwierano: sierpnia 14, 2025, [https://en.wikipedia.org/wiki/Immediate_mode_\(computer_graphics\)](https://en.wikipedia.org/wiki/Immediate_mode_(computer_graphics))
28. Introduction to the Render Graph in Unity 6 - YouTube, otwierano: sierpnia 14, 2025, <https://www.youtube.com/watch?v=U8PygjYAF7A>
29. Get Started With Render Graph In Unity 6 - 80 Level, otwierano: sierpnia 14, 2025, <https://80.lv/articles/introduction-to-urp-s-render-graph-in-unity-6>
30. Analyze a render graph in URP - Unity - Manual, otwierano: sierpnia 14, 2025, <https://docs.unity3d.com/6000.1/Documentation/Manual/urp/render-graph-view.html>
31. GDC 2024 Work graphs and draw calls – a match made in heaven ..., otwierano: sierpnia 14, 2025, <https://gpuopen.com/learn/gdc-2024-workgraphs-drawcalls/>
32. GPU Driven Rendering Overview - Vulkan Guide, otwierano: sierpnia 14, 2025, https://vkguide.dev/docs/gpudriven/gpu_driven_engines/
33. D3D12 Work Graphs - DirectX Developer Blog, otwierano: sierpnia 14, 2025, <https://devblogs.microsoft.com/directx/d3d12-work-graphs/>
34. Advancing GPU-Driven Rendering with Work Graphs in Direct3D 12 - NVIDIA Developer, otwierano: sierpnia 14, 2025, <https://developer.nvidia.com/blog/advancing-gpu-driven-rendering-with-work-graphs-in-direct3d-12/>
35. Work Graphs in Direct3D 12: A Case Study of Deferred Shading | NVIDIA Technical Blog, otwierano: sierpnia 14, 2025, <https://developer.nvidia.com/blog/work-graphs-in-direct3d-12-a-case-study-of-deferred-shading/>
36. GDC 2024 - GPU Work Graphs: Welcome to the Future of GPU Programming - YouTube, otwierano: sierpnia 14, 2025, <https://www.youtube.com/watch?v=QQP6-JF64DQ>
37. Advanced Graphics Programming in Unreal, part 2 | by William Mishra-Manning - Medium, otwierano: sierpnia 14, 2025, <https://medium.com/@manning.w27/advanced-graphics-programming-in-unreal-part-2-abf8237491c1>
38. Movie Render Graph (Beta) - Unreal Engine Public Roadmap, otwierano: sierpnia 14, 2025, <https://portal.productboard.com/epicgames/1-unreal-engine-public-roadmap/c/1694-movie-render-graph-beta->
39. How to render layers using the new Unreal Engine Movie Render Graph - YouTube, otwierano: sierpnia 14, 2025, <https://www.youtube.com/watch?v=8o2yaZzfHCA>
40. Using Movie Render Graph - Tutorial & Course Discussions - Unreal Engine Forums, otwierano: sierpnia 14, 2025, <https://forums.unrealengine.com/t/course-using-movie-render-graph/1819845>
41. Tutorial: Using the Movie Render Graph to Render in Layers - Unreal Engine Forums, otwierano: sierpnia 14, 2025, <https://forums.unrealengine.com/t/tutorial-using-the-movie-render-graph-to-render-in-layers/1843763>
42. Short introduction to Unity's Render Graph | by Kajetan Radulski | Medium,

- otwierano: sierpnia 14, 2025,
<https://medium.com/@hesmeron/short-introduction-to-unitys-render-graph-ed6d66e87d5c>
43. Frame Graph Basic Concepts and Getting Started | Babylon.js Documentation, otwierano: sierpnia 14, 2025,
<https://doc.babylonjs.com/features/featuresDeepDive/frameGraph/frameGraphBasicConcepts/>
 44. Compositor - OGRE - GitHub Pages, otwierano: sierpnia 14, 2025,
<https://ogrecave.github.io/ogre-next/api/2.2/compositor.html>
 45. Compositor Scripts - OGRE, otwierano: sierpnia 14, 2025,
https://ogrecave.github.io/ogre/api/13/_compositor_scripts.html
 46. Open Source Game Development Summit: Render Graph, a Data-Oriented Approach, otwierano: sierpnia 14, 2025,
<https://uat.gdcvault.com/play/1028845/Open-Source-Game-Development-Summit>
 47. Render Pipeline Debug | High Definition RP | 10.2.2 - Unity - Manual, otwierano: sierpnia 14, 2025,
<https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@10.2/manual/Render-Pipeline-Debug-Window.html>
 48. Debugging - LearnOpenGL, otwierano: sierpnia 14, 2025,
<https://learnopengl.com/In-Practice/Debugging>
 49. Debug Shader Graph shaders with symbols | by Bertrand Guay Paquet | Toca Boca Tech Blog | Medium, otwierano: sierpnia 14, 2025,
<https://medium.com/toca-boca-tech-blog/debug-shader-graph-shaders-with-symbols-2c66aed62a59>
 50. Introduction to the Frame Debugger - Unity - Manual, otwierano: sierpnia 14, 2025,
<https://docs.unity3d.com/6000.1/Documentation/Manual/FrameDebugger.html>
 51. Metric graphs 101: Graphing anti-patterns - Datadog, otwierano: sierpnia 14, 2025, <https://www.datadoghq.com/blog/anti-patterns-metric-graphs-101/>